Assignment 6: AWS advance networking with VPC and Subnets

Overview

This assignment provides a hands-on experience in designing and implementing a secure, highly available, and scalable network infrastructure using Amazon Virtual Private Cloud (VPC). You will build a multi-tier, multi-AZ network from the ground up, configure routing for public and private subnets, implement layered security controls, and establish private connectivity between VPCs and to AWS services. The objective is to develop a deep, practical understanding of core AWS networking components and best practices.

Deliverables

This is a group assignment. All task should be implemented by student group or it can be implemented individually (**Access AWS academy: Sandbox → Sandbox Environment**). Each student must submit presentation slides or PDF.

1. **Presentation making guidelines**
   - Use professional formatting (legible fonts, consistent colors, organized layout).
   - Ensure clarity: avoid overcrowding slides with text; use bullet points and visuals.
   - Title slide with student name(s), course, and assignment title
   - Provide output screenshots as evidence of work completed.
   - Keep presentations within the time limit (6–10minutes).
   - Be prepared to answer questions about the process and challenges.
2. **Task complementation**: Each AWS task is evaluated based on correctness, completeness, and evidence (e.g., screenshots of output). **(60 points total)**.
3. **Presentation Quality (20 points total)**
   - Clear structure (intro, results, discussion, conclusion)
   - Slides visually organized and professional, for example, adding appropriate title of each task, pictures, transitions, animations Screenshots, included as evidence of each task results with proper description of each task results.
4. **Demonstration (in-class or video recorded) (10 point total)**

- A 9–12 minute walk-through of the slides and discussion of your results.

5. **Professionalism & Delivery (10 points total)**
   - Confident and clear presentation delivery
   - Submit slides (PowerPoint or PDF) to Moodle by the deadline..

6. Clear explanation of each following completed task:

6.1. **Task 1: Create a Custom VPC with Public and Private Subnets:** Understand how to create an isolated VPC network and define public and private subnets within it.

- **Instructions:**
  - ☑ Navigate to the **VPC** service in the AWS Management Console.
  - ☑ In the top-right corner, ensure you are in your desired region (e.g., **N. Virginia (us-east-1)**).
  - ☑ Click **Create VPC**.
  - ☑ Select **VPC only**.
  - ☑ Under **Name tag**, enter VPC1.
  - ☑ For **IPv4 CIDR block**, enter `10.10.0.0/16`.
  - ☑ Leave all other settings as default and click **Create VPC**.
  - ☑ Once the VPC is created, go to **Subnets** in the left-hand menu.
  - ☑ Click **Create subnet**.
  - ☑ Select **VPC1** from the **VPC ID** dropdown.
  - ☑ Define your **public subnet**:
    - **Subnet name:** public-subnet
    - **Availability Zone:** Choose any (e.g., us-east-1a).
    - **IPv4 CIDR block:** `10.10.1.0/24`
  - ☑ Click **Add new subnet** to add a second subnet configuration in the same window.
  - ☑ Define your **private subnet**:
    - **Subnet name:** private-subnet
    - **Availability Zone:** Choose the same one as your public subnet (e.g., us-east-1a).
    - **IPv4 CIDR block:** `10.10.2.0/24`

☑. Click **Create subnet** at the bottom of the page.

☑. Select your public-subnet. In the **Details** tab, observe the **Available IPv4 addresses**. It should be **251** (256 total IPs minus 5 reserved by AWS), confirming the address allocation.

6.2. **Task 2: Launch EC2 Instances in Each Subnet:** Deploy public and private compute resources within your custom VPC.

- **Instructions:**

  ☑ Navigate to the **EC2** service.

  ☑ Click **Launch Instance**.

  ☑ **Launch EC2-1 (Public Instance):**

  - **Name:** `EC2-1`
  - **Application and OS Images (AMI):** Select **Amazon Linux 2 AMI** (Free tier eligible).
  - **Instance type:** Select **t2.micro** (Free tier eligible).
  - **Key pair (login):** select an existing key pair.
  - **Network settings:** Click **Edit**.
    - **VPC:** Select **VPC1**.
    - **Subnet:** Select **public-subnet**.
    - **Auto-assign public IP:** Select **Enable**.
  - **Firewall (security groups):**
    - Select **Create a new security group**.
    - **Security group name:** `public-sg`
    - **Description:** Allows SSH and ICMP
    - **Inbound rules:**
      - Rule 1: Type **SSH**, Source **My IP** (This is more secure. If your IP changes, use `0.0.0.0/0`, but be aware this is less secure).
      - Rule 2: Click **Add security group rule**. Type **All ICMP - IPv4**, Source **Custom** `10.10.0.0/16` (This allows ping from *anywhere* inside VPC1).

  ☑. Click **Launch instance**.

6.3. **Task 3: Enable Internet Access for Public Subnet:** Connect your VPC to the Internet using an Internet Gateway (IGW) to allow access for public workloads.

- **Instructions:**

  ☑ Navigate to the **VPC** service.

  ☑ In the left-hand menu, click **Internet Gateways**.

  ☑ Click **Create internet gateway**.

  ☑ **Name tag:** IGW1. Click **Create internet gateway**.

  ☑ Select IGW1 from the list. Click **Actions** -> **Attach to VPC**.

  ☑ Select **VPC1** from the dropdown and click **Attach internet gateway**.

  ☑ In the left-hand menu, click **Route Tables**.

  ☑ You will see a default route table for VPC1. Select it.

  ☑ **Name** it `public-rt` for clarity.

  ☑ With public-rt selected, click the **Routes** tab below.

  ☑ Click **Edit routes**.

  ☑ Click **Add route**.

  - **Destination:** `0.0.0.0/0`
  - **Target:** Select **Internet Gateway** and then choose **IGW1**.

  ☑ Click **Save changes**.

  ☑ Now, you must associate this route table with your public subnet. Click the **Subnet Associations** tab.

  ☑ Click **Edit subnet associations**.

  ☑ Check the box for **public-subnet** and click **Save associations**.

  ☑ **Verify Internet Access (EC2-1 via browser)**

  - Navigate to EC2 service.
  - Select EC2-1 instance. Then click on connect.
  - Select EC2 Instance Connect option
  - Then click on Connect.
  - Wait for a minute AWS SSH client terminal of EC2-1 will be open.
  - Test the connection via command ping `8.8.8.8`

- After this, generate a key from EC2-1 terminal by entering command:
  `ssh-keygen -t rsa -f ~/.ssh/mykey`
- See the key via command: **ls -l ~/.ssh**
- Now display the key in terminal using command: **cat ~/.ssh/mykey.pub**
- The key should be look like: "`ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQCt5UZNBRXcycgAznrD5mbBic42yfP8zVhc5W5NMvJuuV8Uw8khTuAeEoK7AUYzWr3QjR6sak0JzHKG1I0wsI6xgGH41i8C3JAhL9w6jw8xkX/zrKg/qVUzByuxLdqy5jl2DA9+5928dEwpWq3vfUezqWJyNwTf7qco9Fl+hLa8ZYb/8Mc3RdLOy05mWjKNk3/4oBAu4qt10mMPqH6lO7xrY/zCjK7zUrPMOZrvJtaTg3OlOxtRR+EWb8SXCQid2Lg10KcFqMw8eGjc85cA2WWcIZOq7CT6tdFGVpNsqdvVNBpn/zBExPZnbS7I40TEbR/TSrj/+YSmqZjfOpD1BMO/H42rkTXpAVNgoLRCspG2rXBh79U8rPlTsymT0W00CK9Ubf9mlEGIc62HnjvcI5Jy4qEhWJN4UHQZVkBBRqxzcvaZKkOpnNjRNs6JhudIXzqF5X8KSd/ilcAD8KmV/kJi/ddI5oZu/NTC1xOiwBdkinhorLnAnLOm5pkGwgYT83E= ec2-user@ip-10-10-1-212.ec2.internal`"
- Copy the full-key some were to use it later.

6.4. **Task 4: Provide Outbound Internet Access for Private Subnet (NAT Gateway):** Allow private instances (like EC2-2) to access the Internet for updates without exposing them to inbound connections.

- **Instructions:**
  - **Launch EC2-2 (Private Instance):**
    - ☑ Click **Launch Instance** again.
    - ☑ **Name:** `EC2-2`
    - ☑ **AMI:** Select **Amazon Linux 2 AMI**.
    - ☑ **Instance type:** Select **t2.micro**.
    - ☑ **Key pair (login):** Select the **same key pair** you used for EC2-1.
    - ☑ **Network settings:** Click **Edit**.
      - **VPC:** Select **VPC1**.
      - **Subnet:** Select **private-subnet**.
      - **Auto-assign public IP:** Select **Disable**.
    - ☑ **Firewall (security groups):**

☑ Select **Create a new security group**.
☑ **Security group name:** `private-sg`
☑ **Description:** Allows SSH and ICMP from within VPC

☑ **Inbound rules:**
- Rule 1: Type **All ICMP - IPv4**, Source **Custom** `10.10.0.0/16` (Allows ping from VPC1).
- Rule 2: Click **Add security group rule**. Type **SSH**, Source **Custom** `10.10.1.0/24` (Allows SSH *only* from your public subnet).

☑ **Go to Advance**
- Navigate to User data. Copy this code and paste it on User data field.

```
#!/bin/bash
mkdir -p /home/ec2-user/.ssh
echo "PASTE_YOUR_mykey.pub_HERE" >> /home/ec2-
user/.ssh/authorized_keys
chown -R ec2-user /home/ec2-user/.ssh
chmod 700 /home/ec2-user/.ssh
chmod 600 /home/ec2-user/.ssh/authorized_keys
```

- Replace PASTE_YOUR_mykey.pub_HERE with your actual key which you have created with EC2-1 user.
- After click on Launch instance. Wait ~1 minute after it starts and pass checks.

☑ Now navigate to the **VPC** service.
☑ In the left-hand menu, click **NAT Gateways**.
☑ Click **Create NAT gateway**.

- **Name:** NAT1
- **Subnet:** Select **public-subnet** (A NAT Gateway must reside in a public subnet).
- **Connectivity type: Public**.
- **Elastic IP allocation:** Click **Allocate Elastic IP**.

☑ Click **Create NAT gateway**. (Provisioning may take a few minutes).
☑ While it's provisioning, go to **Route Tables** and click **Create route table**.

- **Name:** `private-rt`
- **VPC:** Select **VPC1**.

☑. Click **Create route table**.
☑. Select your new private-rt from the list.
☑. In the **Routes** tab, click **Edit routes**.
☑. Click **Add route**.
  - Destination: `0.0.0.0/0`
  - **Target:** Select **NAT Gateway** and then choose **NAT1**.

☑. Click **Save changes**.
☑. Click the **Subnet Associations** tab for private-rt.
☑. Click **Edit subnet associations**.
☑. Check the box for **private-subnet** and click **Save associations**.

☑. **Verify Outbound Access:**

- Once again go to the EC2-1 AWS browser terminal.
- Login in to EC2-2 user through EC2-1 by using command:
  - `ssh -i ~/.ssh/mykey ec2-user@<private IP>`
- You will successfully login to EC2-2 user via NET gateway.
- test its internet access: curl google.com

☑ You should get an HTML response. ping `8.8.8.8` will also work. This confirms EC2-2 can reach the internet via the NAT Gateway.

6.5. **Task 5: Peer Two VPCs:** Enable private, direct communication between two VPCs as if they were on the same network.

- **Instructions:**

☑ **Create VPC2:**

☑ Go to the **VPC** service -> **Create VPC**.

☑ Select **VPC only**. **Name:** VPC2. **IPv4 CIDR:** `8.8.0.0/16.`

☑ Click **Create VPC.**

☑ **Create Subnet for VPC2:**

- Go to **Subnets** -> **Create subnet**.
- **VPC ID: VPC2**. **Subnet name:** subnet-8. **IPv4 CIDR:** 8.8.8.0/24.
- Click **Create subnet**.

☑ Go to **Route Tables** and click **Create route table**.

- **Name:** `main-route-table`
- **VPC:** Select **VPC2**.
- Click **Create route table**.

☑. **Launch EC2-8 in VPC2:**

- Go to **EC2** -> **Launch Instance**.
- **Name:** `EC2-8`. **AMI: Amazon Linux 2**. **Type: t2.micro**.
- **Key pair:** Select the **same key pair** as before.
- **Network settings (Edit):**
  - **VPC: VPC2**.
  - **Subnet: subnet-8**.
  - **Auto-assign public IP: Disable**.
- **Firewall (Security Groups):**
  - Select **Create a new security group**.
  - **Security group name:** `vpc2-sg`
  - **Inbound rules:** Add rule: Type **All ICMP - IPv4**, Source **Custom** `10.10.0.0/16` (VPC1's CIDR).
- Click **Launch instance**.

☑ **Create Peering Connection:**

- Go to the **VPC** service -> **Peering connections** -> **Create peering connection**.
- **Name:** `vpc1-vpc2-peer`.
- **Requester VPC: VPC1**.
- **Accepter VPC: VPC2**.
- Click **Create peering connection**.

☑ **Accept Peering Connection:**

- Select the new "Pending acceptance" connection.
- Click **Actions** -> **Accept request**. Confirm.

☑ **Update All Route Tables (Critical Step):**

- Go to **Route Tables**.
- **For VPC1 (public-rt):**
  - Select public-rt. **Routes** tab -> **Edit routes** -> **Add route**.
  - **Destination:** `8.8.0.0/16` (VPC2's CIDR).
  - **Target:** Select **Peering Connection** and choose vpc1-vpc2-peer. Save.
- **For VPC1 (private-rt):**
  - Select private-rt. **Routes** tab -> **Edit routes** -> **Add route**.
  - **Destination:** `8.8.0.0/16`.
  - **Target:** Select **Peering Connection** and choose vpc1-vpc2-peer. Save.
- **For VPC2 (main Route Table):**
  - Select the main route table for **VPC2**.
  - **Routes** tab -> **Edit routes** -> **Add route**.
  - **Destination:** `10.10.0.0/16` (VPC1's CIDR).
  - **Target:** Select **Peering Connection** and choose vpc1-vpc2-peer. Save.
- Click the **Subnet Associations** tab for main-route-table.
  - Click **Edit subnet associations**.
  - Check the box for subnet-8 and click **Save associations**.

☑ Test Peering:

- SSH into **EC2-1** (in VPC1).
- Find the **Private IPv4 address** of **EC2-8** (e.g., 8.8.8.x).
- From EC2-1, ping EC2-8: ping <EC2-8-Private-IP>.
- The ping should be successful, proving the peering connection and routes are working.

6.6. **Task 6: Secure the VPC (Firewalls and Endpoints):** Harden your VPC by applying layered security (Security Groups and NACLs) and prepare for private S3 access.

- **Instructions:**
  - ☑ Refine Security Groups:
  - ☑ Go to EC2 -> Security Groups.
  - ☑ Select public-sg (for EC2-1). Click **Edit inbound rules**.
    - **Add** a rule: Type **HTTP**, Source **0.0.0.0/0**. (This allows web server traffic).
    - Ensure your **All ICMP - IPv4** rule also allows the peer VPC: Add 8.8.0.0/16 as a source if not already covered.
    - Select private-sg (for EC2-2).
    - Ensure its **All ICMP - IPv4** rule also allows `8.8.0.0/16`.
    - Select vpc2-sg (for EC2-8).
    - Ensure its **All ICMP - IPv4** rule allows 10.10.0.0/16.
  - ☑ Review Network ACLs (NACLs):
    - Go to VPC -> Network ACLs.
    - Select the **default NACL** for **VPC1**.
    - Review the Inbound Rules and Outbound Rules.
  - ☑ **Observation:** By default, the rules are 100 ALL Traffic ALLOW and * ALL Traffic DENY. This stateless configuration allows all traffic. In a real-world scenario, you would tighten these rules to only allow specific ports (e.g., 22, 80, 443, ICMP) in *both* directions (inbound and outbound) and deny all else.
  - ☑ For this lab, you can leave the default NACLs as "ALLOW ALL" and rely on your stateful Security Groups as the primary defense.
  - ☑ Create a VPC Endpoint for S3:

    - Go to VPC -> Endpoints -> Create endpoint.
    - **Name:** `s3-gateway-endpoint`
    - Service category: AWS services.
    - Search for "s3" and select the service with **Type = Gateway**. (It will look like com.amazonaws.[region].s3).
    - **VPC:** Select **VPC1**.
    - **Route tables:** Check the box for your **private-rt** (the one associated with private-subnet).
    - Click **Create endpoint**. This creates a private route from your private subnet directly to S3.

6.7. **Task 7: Verify Private S3 Access through VPC Endpoint:** Demonstrate secure, private access to Amazon S3 from an isolated instance without using a NAT Gateway or the public internet.

- **Instructions:**

☑ **Create an S3 Bucket:**

- Navigate to the **S3** service.
- Click **Create bucket**.
- **Bucket name:** `my-vpc-s3-demo-bucket-[your-unique-id]` (e.g., add your initials/date).
- Ensure **Block all public access** remains **checked**.
- Click **Create bucket**.
- Inside your new bucket, click **Upload** -> **Add files**.
- Create a simple file named demo.txt or index.html on your computer and upload it.

☑ **Attach Role to EC2-2:**

- Go back to the **EC2** service -> **Instances**.
- Select **EC2-2** (your private instance).
- Click **Actions** -> **Security** -> **Modify IAM role**.
- Choose existing **EMR_EC2_DeafulRrole** from the dropdown.
- Click **Update IAM role**. (It may take a moment to apply).

☑ **Test S3 Access (Success):**

- SSH into **EC2-1** and then "hop" into **EC2-2** (as you did in Task 4).
- On **EC2-2**, run the following AWS CLI command:
  - `aws s3 ls`
- You should see a list of your S3 buckets, including my-vpc-s3-demo-bucket....
- Now, list the contents of your bucket:
  - `aws s3 ls s3://my-vpc-s3-demo-bucket-[your-unique-id]`
- You should see demo.txt listed. This request worked *privately* through the VPC Endpoint.

- Remember to delete all resources you have created so far to avoid incurring cost.

6.8. **Task 8: Deploy VPC using Terraform:** Understand how to deploy a full-stack environment (VPC, subnets, gateways, and instances) automatically using Infrastructure as Code (IaC) with Terraform. This task replicates the manual work from Tasks 1 to 4 using a declarative script.

- **Instructions:**
  - Navigate to EC2 services and click on cloudshell icon to open the AWS cloudshell.
  - **Prerequisite -** Install Terraform through AWS cloudshell by following command.
    - `sudo yum install -y yum-utils`
    - `sudo yum-config-manager --add-repo https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo`
    - `sudo yum install terraform -y`
    - `terraform version`
  - **Prepare Project Directory:**

- Create a new folder for your project (e.g., terraform-vpc-lab).
- Inside this folder, create the files: main.tf by following command
  - nano main.tf
- Copy the following content to your main.tf file (this is the file you need to edit):

```
provider "aws" {
region = "us-east-1"
}
resource "aws_vpc" "test" {
cidr_block = "10.10.0.0/16"
tags = {
Name = "VPC1"
   }
}
resource "aws_subnet" "public" {
vpc_id = aws_vpc.test.id
cidr_block = "10.10.1.0/24"
availability_zone = "us-east-1a"
map_public_ip_on_launch = true
tags = {
Name = "public-subnet"
  }
}
```

- Then press `Ctrl+Z`.
- Then will ask to save.
- Input `Y` for yes.
- Press Enter
- The `main.tf` file will saved.

- **Initialize Terraform:** Run the following command to download the necessary AWS provider plugins:
  - `terraform fmt`
  - `terraform init`
  - `terraform validate`

- **Plan the Deployment:** Run the plan command. This will check your script and show you all the resources (VPC, subnets, instances, etc.) that will be created.
  - `terraform plan`
  - Review the output to ensure there are no errors.

- **Apply the Configuration:** Run the apply command to build the infrastructure.

```
terraform apply
```

- Terraform will show you the plan again. Type yes and press **Enter** to approve the deployment.
- Wait for 3-5 minutes as Terraform provisions all the resources. When it's finished, it will display the "Outputs".

- **Verify in Console:**

  - Go to the AWS Management Console.
  - Navigate to the **VPC** services. You will see all the new resources, such as VPC1-Terraform, public subnet.

- **Destroy Resources (CRITICAL):**

- To avoid incurring costs, you **must** destroy all resources when you are finished.
- In your terminal, run the destroy command:

  - `terraform destroy`
  - Type yes and press **Enter** to confirm. Terraform will now tear down and delete all the resources it created.
  - After that you can verify in the VPC services by clicking refresh button. You will created VPC and subnet is deleted.

6.9. Task 9: Reflestion Question

What are the advantages of using Terraform to automate networking infrastructure in Amazon Web Services instead of configuring resources manually through the AWS Console?

----------END----------

Dr. Rohit Kumar Kasera
Lecturer